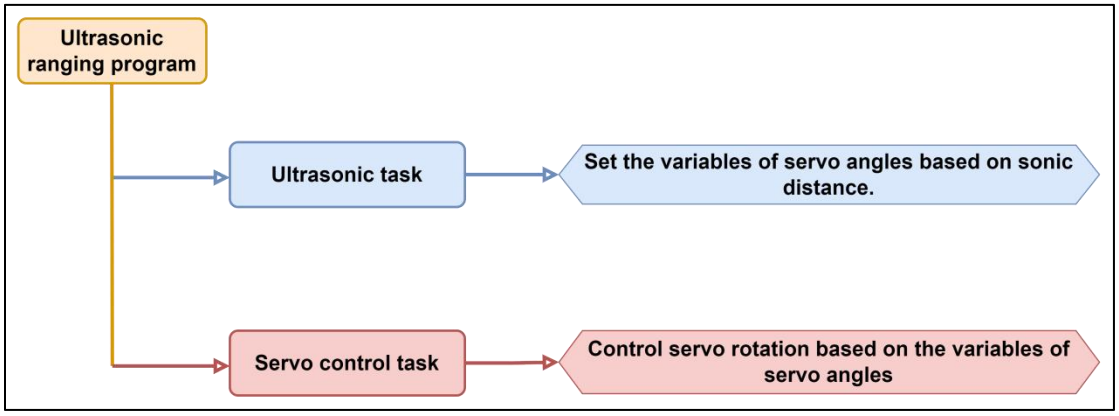
5.3 Ultrasonic Ranging

In this section, you will learn how to use the glowing ultrasonic module to test the obstacle distance, achieve the function of opening and closing the mechanical claw and control the color change of RGB lights.

1. Program Flowchart



2. Ultrasonic Sensor

This is a glowing ultrasonic ranging module. The module adopts I2C communication interface, which can read distance measured by ultrasonic sensor through I2C communication.

2.1 Working Principle

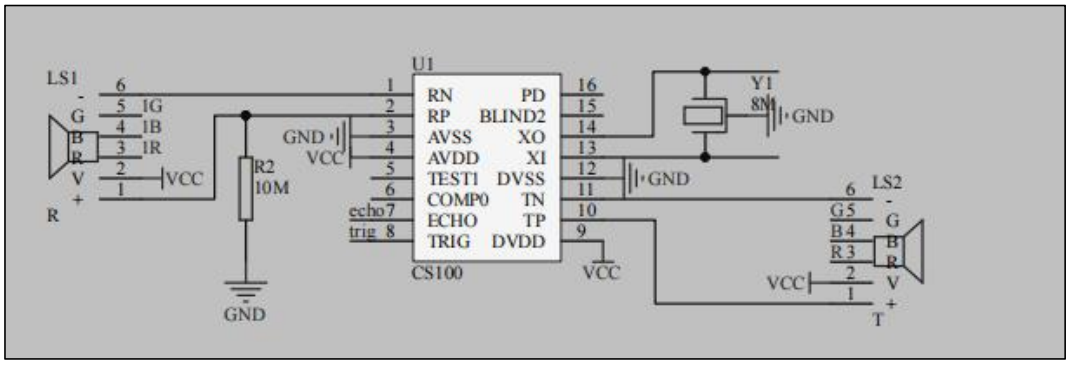
The module will automatically transmit 8 square waves at 40khz during

sensor ranging, and detect whether there is a signal return. If there is a signal returned, output a high voltage level. And the duration of high voltage level is the time that the ultrasonic wave takes from transmission to return.

Formula: measurement distance=(high level time*sound speed(340M/S))/2.

2.2 Schematic Diagram of Sensor

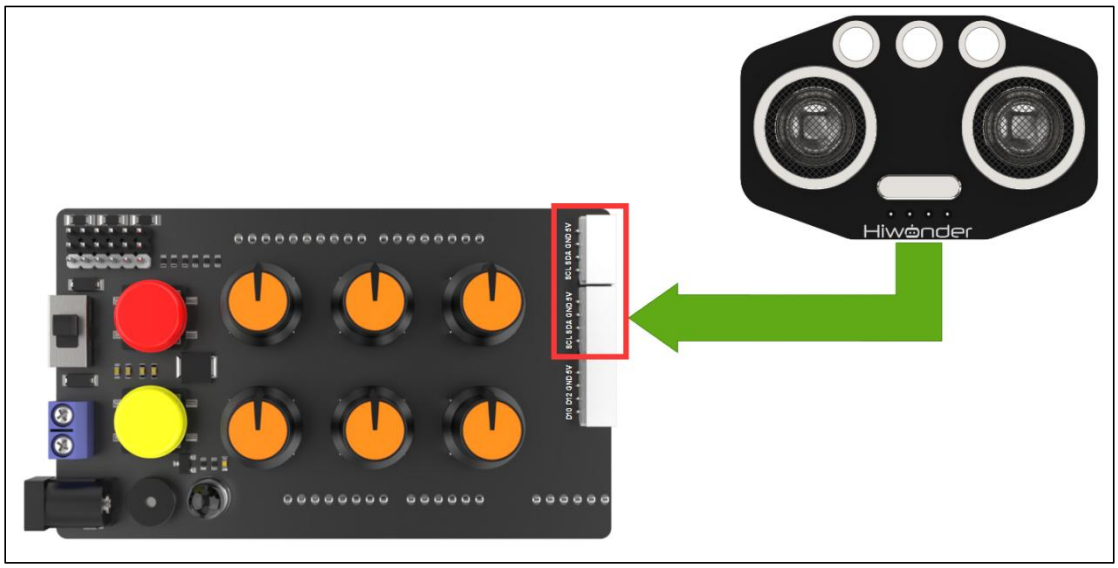
The sensor is controlled by CS100 chip. The chip pins TP and TN can transmit 8 square waves at 40khz. And then the echo signal is detected by chip pins RP and RN. The distance value is calculated through the formula “measurement distance=(high level time*sound speed(340M/S))/2”.



Supply Voltage: 5V
Working Current: 2mA
Detection Distance: 2cm - 400cm

2.3 Sensor Wiring

Connect the ultrasonic module to any I2C interface through 4Pin cable.



3. Program Download

Note:

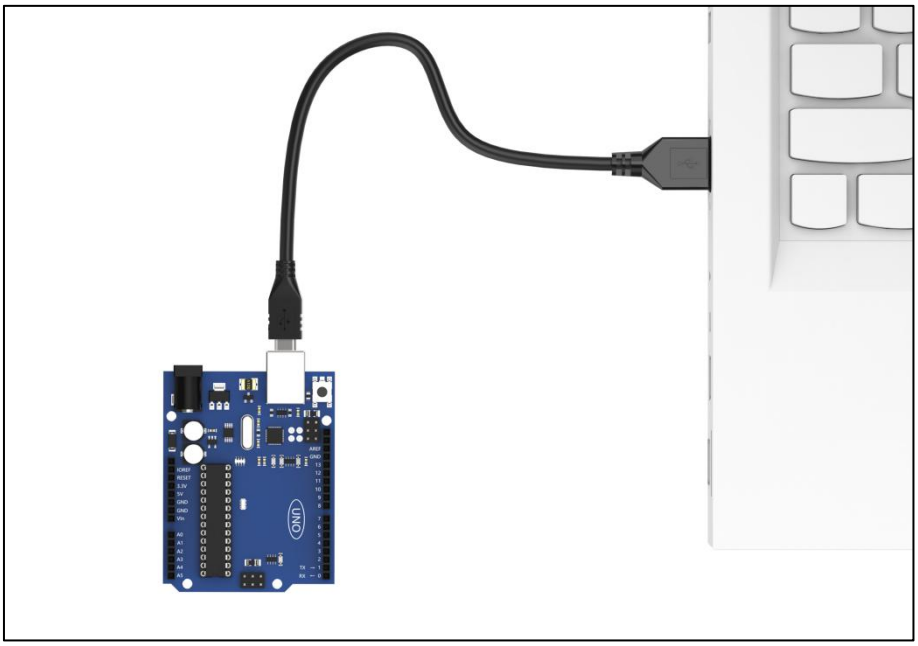
◆ Remove the Bluetooth module before downloading the program, otherwise the serial port conflict will lead to program download failure.

◆ Please turn the switch of the battery box to “OFF” when connect Type-B download cable to avoid the cable touch the power pins of expansion board by mistake, which may cause a short circuit.

1)Locate “MiniArm_Ultrasound\MiniArm_Ultrasound.ino” program file in the same path as this document.

名称	修改日期
MiniArm_Ultrasound.ino	2024/2/26 14:13
tone.h	2024/2/26 14:13
Ultrasound.cpp	2024/2/26 14:13
Ultrasound.h	2024/2/26 14:13

2)Connect Arduino to the computer through UNO Type-B cable.



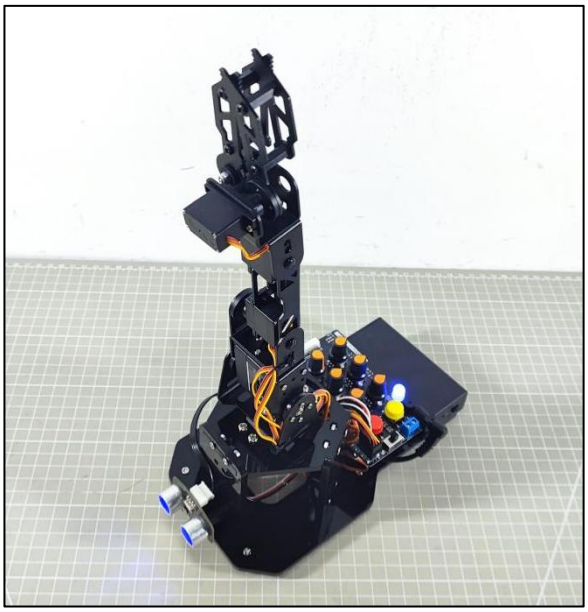
3)Click “Select Development Board” option. The software will automatically test current Arduino serial port, then click to connect.

4)Click to download the program to Arduino, then wait for download to complete.

4. Program Outcome

After powering on, the robotic arm will return to the initial position. Meanwhile, the RGB lights on the expansion board and glowing ultrasonic

module will emit blue.

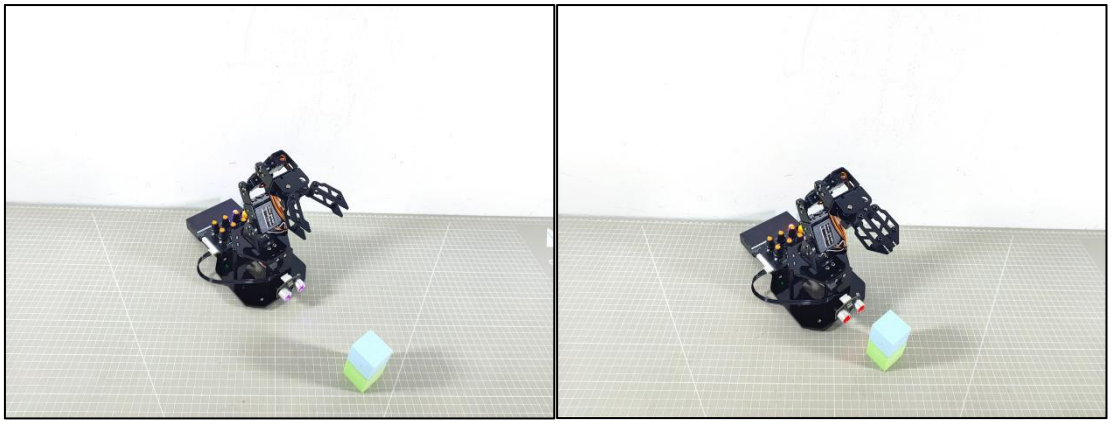


Place the obstacle directly in front of the ultrasonic module and slowly move it closer.

When the distance is greater than 20cm, the mechanical claw will open and the RGB lights on the ultrasonic module and expansion board will emit blue.

When the distance is less than 10cm and greater than 20cm, the obstacle will close to the robotic arm and the mechanical claw will close slowly. Meanwhile, the color of RGB lights on the ultrasonic module and the expansion board will change from blue to purple.

When the distance is less than 10cm, the mechanical claw will close. And the RGB lights on the ultrasonic module and the expansion board will emit red.



5. Program Brief Analysis

5.1 Import Library File

```
#include <FastLED.h> //import LED library
#include <Servo.h> //import servo library
#include "tone.h" //tone library
#include "Ultrasound.h" //import ultrasonic library
#include <EEPROM.h>
```

Import the library files required for this program, including the RGB control library, servo control library, buzzer library and ultrasonic control library.

5.2 Define Pins and Create Objects

1)The EEPROM start flag, the start and data addresses of the servo deviation data, and the length of the servo deviation data are defined.

```
#define EEPROM_START_FLAG "HIWONDER"
#define EEPROM_SERVO_OFFSET_START_ADDR 0u
#define EEPROM_SERVO_OFFSET_DATA_ADDR 16u
#define EEPROM_SERVO_OFFSET_LEN 6u
```

2)Define the tones of the two buzzers and the Arduino pins connected to the hardware: mainly consist of six servo pins, one buzzer pin and one RGB light pin.

```
const static uint16_t DOC5[] = { TONE_C5 };
const static uint16_t DOC6[] = { TONE_C6 };

/* pins definition */
const static uint8_t servoPins[6] = { 7, 6, 5, 4, 3, 2 };
const static uint8_t buzzerPin = 11;
const static uint8_t rgbPin = 13;
```

3)Create the objects of the RGB light, servo and ultrasonic sensor. Meanwhile, variables controlled by the servo are created. For example, the “extended_func_angles” array is used for storing desired angle for each servo, and the “servo_angles” array is used for storing the actual angle of servo with range from 0 to 180. In addition, angle limits are set for each joint of the robotic arm.

```
//servo control object
Servo servos[6];

//create ultrasonic object
Ultrasound ul;
/* Angle values used in secondary development routines */
static uint8_t extended_func_angles[6] = { 73,10,161,57,90,90 };
/* Limitations on joint angles */
const uint8_t limt_angles[6][2] = {{0,82},{0,180},{0,180},{0,180},{0,180},{0,180}};
/* The angle value of the actual control of the servo */
static uint8_t servo_angles[6] = { 73,10,161,57,90,90 };
static int8_t servo_offset[5];
static uint8_t servo_expect[5];
static uint8_t eeprom_read_buf[16];
```

For the desired value and the actual value, you can see the “servo_control” function called in the loop main function.

If you want the servo to approach to the target position gradually, call this line of code “servo_angles[i] = servo_angles[i] * 0.9 + extended_func_angles[i] * 0.1” in the control task function.

This causes the servo to move each time by adding a portion of the servo pulse-width of the desired value to its previous position. After each movement, the current new data will be reassigned. This enables the robotic arm to stop moving when the target position is reached.

```
for (int i = 0; i < 5; ++i) {
  if(servo_angles[i] > extended_func_angles[i])
  {
    servo_angles[i] = servo_angles[i] * 0.9 + extended_func_angles[i] * 0.1;
    if(servo_angles[i] < extended_func_angles[i])
      servo_angles[i] = extended_func_angles[i];
  }else if(servo_angles[i] < extended_func_angles[i])
  {
    servo_angles[i] = servo_angles[i] * 0.9 + (extended_func_angles[i] * 0.1 + 1);
    if(servo_angles[i] > extended_func_angles[i])
      servo_angles[i] = extended_func_angles[i];
  }
```

4)Define servo-related variables and EEPROM read buffer.

```
static int8_t servo_offset[5];
static uint8_t servo_expect[5];
static uint8_t eeprom_read_buf[16];
```

“servo_offset” denotes the servo’s deviation, while “servo_expect” represents the expected value of the servo accounting for this deviation.

5)Define the variable used for controlling buzzer, and declare the task function to execute different control task.

```
//servo control
static void servo_control(void);
//ultrasonic task
static void ultrasound_task(void);
static void read_offset(void);
```

“tune_num” variable means the tone notation; “tune_beat” variable means rhythm; “servo_control” function is used for controlling servo; “ultrasound_task” function is used for reading ultrasonic sensor data.

5.3 Initialization Settings

1)The “setup()” function is used for initializing the relevant hardware devices. First is the serial port, set the baud rate for communication to 115200 and the read data timeout to 500ms.

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  // set the read data timeout of serial port
  Serial.setTimeout(500);
```

2)Assign the servo IO port, use the FastLED library to initialize the RGB light on the expansion board and connect it to the RGB pin. Set the RGB color to green through “rgbs[0] = CRGB(0, 255, 0)”. Finally, use the “FastLED.show” function to display the set color.

```
// assign servo IO port
for (int i = 0; i < 6; ++i) {
  servos[i].attach(servoPins[i],500,2500);
}

//initialize the RGB light and control it
FastLED.addLeds<WS2812, rgbPin, GRB>(rgbs, 1);
rgbs[0] = CRGB(0, 0, 100);
FastLED.show();
```

3)Set the buzzer interface to output mode and call the “tone()” function to make the buzzer sound. Call “noTone()” function to stop sounding the buzzer after a delay of 100ms.

```
//initialize buzzer and beep one time
pinMode(buzzerPin, OUTPUT);
tone(buzzerPin, 1000);
delay(100);
noTone(buzzerPin);
```

5.4 Read Deviation Angle

```
read_offset();
//read deviation angle
```

1)Firstly, get the start flag of the deviation value in memorizer and store it in the “eeprom_read_buf” read buffer.

```
for (int i = 0; i < 16; ++i) {
  eeprom_read_buf[i] = EEPROM.read(EEPROM_SERVO_OFFSET_START_ADDR + i);
}
```

2)Determine whether the start flag bit in the memorizer is correct. If it is correct, the “eeprom_read_buf” buffer will be cleared. The serial port prints the “read offset” string, and then starts to get the deviation value.

```
if (strcmp(eeprom_read_buf, EEPROM_START_FLAG) == 0) {
  memset(eeprom_read_buf,  0 , sizeof(eeprom_read_buf));
  Serial.println("read offset");
```

3)Get 5 servo offset value in the memorizer and store it to the read buffer. Use “memcpy” to copy the data in the read buffer to the “servo_offset” variable.

Since there are positive and negative deviation values, and the “eeprom_read_buf” variable is an unsigned variable, we convert the deviation data from an unsigned number to a signed number by “memcpy”.

```
  for (int i = 0; i < 5; ++i) {
    eeprom_read_buf[i] = EEPROM.read(EEPROM_SERVO_OFFSET_DATA_ADDR + i);
  }
  memcpy(servo_offset , eeprom_read_buf , 6);
```

5.5 Ultrasonic Detection

After initialization is complete, the next step is to call the ultrasonic task function “ultrasound_task” within the main “loop” function. The function is used to implement distance measurement functionality. It first defines the variable “last_tick”, which is mainly used to calculate the time intervals.

```
// Ultrasonic task
void ultrasound_task(void)
{
  static uint32_t last_ul_tick = 0;
  // delay 100ms
  if (millis() - last_ul_tick < 100) {
    return;
  }
  last_ul_tick = millis();

  // Obtain ultrasonic distance
  int dis = ul.Filter();
  // Serial.println(dis);
```

◆ Next, check if 100ms have passed since the last call to the "ultrasound_task" function. If it passed, the following code is executed. Otherwise, the function returns directly. The "millis" function is the time since the Arduino board started. In this way, the function ensures that the task of detecting the distance is performed every 100ms.

◆ Then, get filtered ultrasonic measurements through the "ul.Filter" function, and store the outcome in the variable "dis".

5.6 Execution Feedback

1)If the detected obstacle distance is greater than 200mm, control the servo No.1~5 to rotate to the 180° to open the mechanical claw. Meanwhile, control the RGB lights on the glowing ultrasonic module and the expansion board emit blue through "rgbs" and "ul.color" function.

"rgbs.r" "rgbs.g" "rgbs.b" represent the brightness values of the RGB primary colors for the lights on the expansion board. Meanwhile, "ul.color（0,0,255,0,0,255）" represents the brightness values of the RGB primary colors for the left and right lights of the glowing ultrasonic.

```
// If it is greater than 200mm
if(dis >= 200)
{
  //Open your claws
  extended_func_angles[0] = 0;
  // RGB light blue
  rgbs[0].r = 0;
  rgbs[0].g = 0;
  rgbs[0].b = 255;
  FastLED.show();
  // Luminescent ultrasonic blue
  ul.Color(0,0,255,0,0,255);
```

2)If the distance of the obstacle is detected to be less than 50mm, then control the servo 1~5 to rotate to the 0° position. That is close the mechanical claw. Meanwhile, control the RGB lights on the glowing ultrasonic module and the expansion board to display red color through the “rgbs” and “ul.color” function.

```
}else if(dis <= 50) //if it is less than 50mm
{
  //close the claw
  extended_func_angles[0] = 82;
  // RGB red
  rgbs[0].r = 255;
  rgbs[0].g = 0;
  rgbs[0].b = 0;
  FastLED.show();
  // glowing ultrasonic red
  ul.Color(255,0,0,255,0,0);
```

3)When the distance of the obstacle is between 50mm and 200mm, the R and B in the color RGB are set by the “map” function according to the change of “dis”. For example, “map(dis,50,200,255,0)” is to map the obstacle distance “dis” to the R element, where the range of “dis” is 50mm~200mm. The brightness range of R is 255~0.

For mapping servo angles based on obstacle distance, you can also refer to the above content. Similarly, it involves mapping obstacle distance to the

range of servo angles (0~180°) for opening and closing the gripper.

```
}else{
  /* the color changes according to the variation. The closer it gets,
  the larger the R (red) and the smaller the B (blue). Conversely, the farther away, the opposite occurs */
  int color[3] = {map(dis , 50 , 200 , 255 , 0),0,map(dis , 50 , 200 , 0 , 255)};
  // The mechanical claw closes as it gets closer and opens as it gets farther away
  uint8_t angles = map(dis , 50 , 200 , 82 , 0);
  extended_func_angles[0] = angles;
  rgbs[0].r = color[0];
  rgbs[0].g = color[1];
  rgbs[0].b = color[2];
  FastLED.show();
  ul.Color(color[0],color[1],color[2],color[0],color[1],color[2]);
```

5.7 Servo Control Task

1)In the servo control task, the same variable “last_tick” is used for delay processing.

```
// servo control task
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();
```

2)Next, get the desired value of the servo through “action_ctl” object. Please note that the desired value at this point is the result of the desired value plus the deviation outcome of servo. Therefore, this desired value is the actual desired value. Finally, the actual desired value of the servo is assigned to the actual control angle. Then pass it to the servo object to actually control the rotation of the servo.

```
  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + action_ctl.extended_func_angles[i] * 0.1;
      if(servo_angles[i] < action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }else if(servo_angles[i] < action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + (action_ctl.extended_func_angles[i] * 0.1 + 1);
      if(servo_angles[i] > action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }
    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

Let's further explain the concept of ideal desired value and actual desired value here. In the "action_ctl" object, "extended_func_angles" is the ideal desired value, which excludes deviation due to servo structure. In the actual project,

servos often have a certain range of deviations due to their mechanical structure. To achieve more accurate control of the servo angle, we calculate the additional angle needed for the servo to reach the ideal position by adjusting for this deviation. This additional angle can be positive, negative, or zero. Then, during actual servo control, add this deviation value to the ideal expected angle, resulting in the actual expected angle.

6. Function Extension

Here demonstrate how to modify the color transition of RGB lights from the original gradient purple to yellow as an illustration. The specific modifications can be referred in the following steps:

1)Locate the mapping instructions in the program that control the RGB lights to change color according to the distance. The R and B in the color RGB are set by the "map" function according to the change of "dis". "map(dis,50,200,255,0)" is to map the obstacle distance "dis" to the R element. "map(dis,50,200,0,255)" is to map the obstacle distance "dis" to the B element.

As a result, as the distance decreased, the proportion of red color

increases while the proportion of blue color decreases, displaying a gradient color that appears purple.

```
/* the color changes according to the variation. The closer it gets,
 the larger the R (red) and the smaller the B (blue). Conversely, the farther away, the opposite occurs */
int color[3] = {map(dis , 50 , 200 , 255 , 0),0,map(dis , 50 , 200 , 0 , 255)};
```

2)To replace the mapped element from the B element to the G element in RGB, it's important to note that the range of color elements is from 0 to 255. Then download the program again. In this way, as the obstacle gets closer, the proportion of red color increases while the proportion of green color decreases, resulting in the RGB lights displaying a gradient color that appears yellow.

```
int color[3] = {map(dis , 50 , 200 , 255 , 0),map(dis , 50 , 200 , 0 , 255),0};
```

You can visit the following link to access information about the RGB color table: https://www.bchrt.com/tools/rgbcolor/.

7. FAQ

Q: since the code was upload, the distance measured by the ultrasound is always 0.

A: please check that you have connected the 4-pin cable to the I2C interface and turn the knob on the expansion board to the initial position.

Q: the distance measured by the ultrasonic sensor is sometimes accurate, and sometimes inaccurate.

A: please use a smooth and flat object for distance measurement.